# Socket


Async, streaming plaintext TCP/IP and secure TLS socket server and client connections for ReactPHP.

The socket library provides re-usable interfaces for a socket-layer server and client based on the `EventLoop` and `Stream` components. Its server component allows you to build networking servers that accept incoming connections from networking clients (such as an HTTP server). Its client component allows you to build networking clients that establish outgoing connections to networking servers (such as an HTTP or database client). This library provides async, streaming means for all of this, so you can handle multiple concurrent connections without blocking.

**Table of Contents**

        * TimeoutConnector
        * UnixConnector
        * FixUriConnector
- Install
- Tests
- License

## Quickstart example

Here is a server that closes the connection if you send it anything:

```
$socket = new React\Socket\SocketServer('127.0.0.1:8080');

$socket->on('connection', function (React\Socket\ConnectionInterface $connection) {
    $connection->write("Hello " . $connection->getRemoteAddress() . "!\n");
    $connection->write("Welcome to this amazing server!\n");
    $connection->write("Here's a tip: don't say anything.\n");

    $connection->on('data', function ($data) use ($connection) {
        $connection->close();
    });
});
```

See also the examples.

Here's a client that outputs the output of said server and then attempts to send it a string:

```
$connector = new React\Socket\Connector();

$connector->connect('127.0.0.1:8080')->then(function (React\Socket\ConnectionInterface $conn
    $connection->pipe(new React\Stream\WritableResourceStream(STDOUT));
    $connection->write("Hello World!\n");
}, function (Exception $e) {
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
});
```

## Connection usage

### ConnectionInterface

The `ConnectionInterface` is used to represent any incoming and outgoing connection, such as a normal TCP/IP connection.

An incoming or outgoing connection is a duplex stream (both readable and writable) that implements React's `DuplexStreamInterface`. It contains additional properties for the local and remote address (client IP) where this connection has been established to/from.

Most commonly, instances implementing this `ConnectionInterface` are emitted by all classes implementing the `ServerInterface` and used by all classes implementing the `ConnectorInterface`.

Because the `ConnectionInterface` implements the underlying `DuplexStreamInterface` you can use any of its events and methods as usual:

```
$connection->on('data', function ($chunk) {
    echo $chunk;
});

$connection->on('end', function () {
    echo 'ended';
});

$connection->on('error', function (Exception $e) {
    echo 'error: ' . $e->getMessage();
});

$connection->on('close', function () {
    echo 'closed';
});

$connection->write($data);
$connection->end($data = null);
$connection->close();
// ...
```

For more details, see the `DuplexStreamInterface`.

**getRemoteAddress()** The `getRemoteAddress(): ?string` method returns the full remote address (URI) where this connection has been established with.

```
$address = $connection->getRemoteAddress();
echo 'Connection with ' . $address . PHP_EOL;
```

If the remote address can not be determined or is unknown at this time (such as after the connection has been closed), it MAY return a `NULL` value instead.

Otherwise, it will return the full address (URI) as a string value, such as `tcp://127.0.0.1:8080`, `tcp://[::1]:80`, `tls://127.0.0.1:443`, `unix://example.sock` or `unix:///path/to/example.sock`. Note that individual URI components are application specific and depend on the underlying transport protocol.

If this is a TCP/IP based connection and you only want the remote IP, you may use something like this:

```
$address = $connection->getRemoteAddress();
```

```php
$ip = trim(parse_url($address, PHP_URL_HOST), '[]');
echo 'Connection with ' . $ip . PHP_EOL;
```

**getLocalAddress()** The `getLocalAddress(): ?string` method returns the full local address (URI) where this connection has been established with.

```php
$address = $connection->getLocalAddress();
echo 'Connection with ' . $address . PHP_EOL;
```

If the local address can not be determined or is unknown at this time (such as after the connection has been closed), it MAY return a `NULL` value instead.

Otherwise, it will return the full address (URI) as a string value, such as `tcp://127.0.0.1:8080`, `tcp://[::1]:80`, `tls://127.0.0.1:443`, `unix://example.sock` or `unix:///path/to/example.sock`. Note that individual URI components are application specific and depend on the underlying transport protocol.

This method complements the `getRemoteAddress()` method, so they should not be confused.

If your `TcpServer` instance is listening on multiple interfaces (e.g. using the address `0.0.0.0`), you can use this method to find out which interface actually accepted this connection (such as a public or local interface).

If your system has multiple interfaces (e.g. a WAN and a LAN interface), you can use this method to find out which interface was actually used for this connection.

## Server usage

### ServerInterface

The `ServerInterface` is responsible for providing an interface for accepting incoming streaming connections, such as a normal TCP/IP connection.

Most higher-level components (such as a HTTP server) accept an instance implementing this interface to accept incoming streaming connections. This is usually done via dependency injection, so it's fairly simple to actually swap this implementation against any other implementation of this interface. This means that you SHOULD typehint against this interface instead of a concrete implementation of this interface.

Besides defining a few methods, this interface also implements the `EventEmitterInterface` which allows you to react to certain events.

**connection event** The `connection` event will be emitted whenever a new connection has been established, i.e. a new client connects to this server socket:

```php
$socket->on('connection', function (React\Socket\ConnectionInterface $connection) {
    echo 'new connection' . PHP_EOL;
```

```
});
```

See also the `ConnectionInterface` for more details about handling the incoming connection.

**error event** The `error` event will be emitted whenever there's an error accepting a new connection from a client.

```
$socket->on('error', function (Exception $e) {
    echo 'error: ' . $e->getMessage() . PHP_EOL;
});
```

Note that this is not a fatal error event, i.e. the server keeps listening for new connections even after this event.

**getAddress()** The `getAddress(): ?string` method can be used to return the full address (URI) this server is currently listening on.

```
$address = $socket->getAddress();
echo 'Server listening on ' . $address . PHP_EOL;
```

If the address can not be determined or is unknown at this time (such as after the socket has been closed), it MAY return a `NULL` value instead.

Otherwise, it will return the full address (URI) as a string value, such as `tcp://127.0.0.1:8080`, `tcp://[::1]:80`, `tls://127.0.0.1:443` `unix://example.sock` or `unix:///path/to/example.sock`. Note that individual URI components are application specific and depend on the underlying transport protocol.

If this is a TCP/IP based server and you only want the local port, you may use something like this:

```
$address = $socket->getAddress();
$port = parse_url($address, PHP_URL_PORT);
echo 'Server listening on port ' . $port . PHP_EOL;
```

**pause()** The `pause(): void` method can be used to pause accepting new incoming connections.

Removes the socket resource from the EventLoop and thus stop accepting new connections. Note that the listening socket stays active and is not closed.

This means that new incoming connections will stay pending in the operating system backlog until its configurable backlog is filled. Once the backlog is filled, the operating system may reject further incoming connections until the backlog is drained again by resuming to accept new connections.

Once the server is paused, no futher `connection` events SHOULD be emitted.

```php
$socket->pause();

$socket->on('connection', assertShouldNeverCalled());
```

This method is advisory-only, though generally not recommended, the server MAY continue emitting `connection` events.

Unless otherwise noted, a successfully opened server SHOULD NOT start in paused state.

You can continue processing events by calling `resume()` again.

Note that both methods can be called any number of times, in particular calling `pause()` more than once SHOULD NOT have any effect. Similarly, calling this after `close()` is a NO-OP.

**resume()** The `resume(): void` method can be used to resume accepting new incoming connections.

Re-attach the socket resource to the EventLoop after a previous `pause()`.

```php
$socket->pause();

Loop::addTimer(1.0, function () use ($socket) {
    $socket->resume();
});
```

Note that both methods can be called any number of times, in particular calling `resume()` without a prior `pause()` SHOULD NOT have any effect. Similarly, calling this after `close()` is a NO-OP.

**close()** The `close(): void` method can be used to shut down this listening socket.

This will stop listening for new incoming connections on this socket.

```php
echo 'Shutting down server socket' . PHP_EOL;
$socket->close();
```

Calling this method more than once on the same instance is a NO-OP.

### SocketServer

The `SocketServer` class is the main class in this package that implements the `ServerInterface` and allows you to accept incoming streaming connections, such as plaintext TCP/IP or secure TLS connection streams.

In order to accept plaintext TCP/IP connections, you can simply pass a host and port combination like this:

```php
$socket = new React\Socket\SocketServer('127.0.0.1:8080');
```

Listening on the localhost address `127.0.0.1` means it will not be reachable from outside of this system. In order to change the host the socket is listening on, you can provide an IP address of an interface or use the special `0.0.0.0` address to listen on all interfaces:

```
$socket = new React\Socket\SocketServer('0.0.0.0:8080');
```

If you want to listen on an IPv6 address, you MUST enclose the host in square brackets:

```
$socket = new React\Socket\SocketServer('[::1]:8080');
```

In order to use a random port assignment, you can use the port `0`:

```
$socket = new React\Socket\SocketServer('127.0.0.1:0');
$address = $socket->getAddress();
```

To listen on a Unix domain socket (UDS) path, you MUST prefix the URI with the `unix://` scheme:

```
$socket = new React\Socket\SocketServer('unix:///tmp/server.sock');
```

In order to listen on an existing file descriptor (FD) number, you MUST prefix the URI with `php://fd/` like this:

```
$socket = new React\Socket\SocketServer('php://fd/3');
```

If the given URI is invalid, does not contain a port, any other scheme or if it contains a hostname, it will throw an `InvalidArgumentException`:

```
// throws InvalidArgumentException due to missing port
$socket = new React\Socket\SocketServer('127.0.0.1');
```

If the given URI appears to be valid, but listening on it fails (such as if port is already in use or port below 1024 may require root access etc.), it will throw a `RuntimeException`:

```
$first = new React\Socket\SocketServer('127.0.0.1:8080');

// throws RuntimeException because port is already in use
$second = new React\Socket\SocketServer('127.0.0.1:8080');
```

> Note that these error conditions may vary depending on your system and/or configuration. See the exception message and code for more details about the actual error condition.

Optionally, you can specify TCP socket context options for the underlying stream socket resource like this:

```
$socket = new React\Socket\SocketServer('[::1]:8080', array(
    'tcp' => array(
        'backlog' => 200,
        'so_reuseport' => true,
        'ipv6_v6only' => true
```

```
    )
));
```

> Note that available socket context options, their defaults and effects of changing these may vary depending on your system and/or PHP version. Passing unknown context options has no effect. The `backlog` context option defaults to `511` unless given explicitly.

You can start a secure TLS (formerly known as SSL) server by simply prepending the `tls://` URI scheme. Internally, it will wait for plaintext TCP/IP connections and then performs a TLS handshake for each connection. It thus requires valid TLS context options, which in its most basic form may look something like this if you're using a PEM encoded certificate file:

```
$socket = new React\Socket\SocketServer('tls://127.0.0.1:8080', array(
    'tls' => array(
        'local_cert' => 'server.pem'
    )
));
```

> Note that the certificate file will not be loaded on instantiation but when an incoming connection initializes its TLS context. This implies that any invalid certificate file paths or contents will only cause an `error` event at a later time.

If your private key is encrypted with a passphrase, you have to specify it like this:

```
$socket = new React\Socket\SocketServer('tls://127.0.0.1:8000', array(
    'tls' => array(
        'local_cert' => 'server.pem',
        'passphrase' => 'secret'
    )
));
```

By default, this server supports TLSv1.0+ and excludes support for legacy SSLv2/SSLv3. As of PHP 5.6+ you can also explicitly choose the TLS version you want to negotiate with the remote side:

```
$socket = new React\Socket\SocketServer('tls://127.0.0.1:8000', array(
    'tls' => array(
        'local_cert' => 'server.pem',
        'crypto_method' => STREAM_CRYPTO_METHOD_TLSv1_2_SERVER
    )
));
```

> Note that available TLS context options, their defaults and effects of changing these may vary depending on your system and/or PHP version. The outer context array allows you to also use `tcp` (and possibly more) context options at the same time. Passing unknown

context options has no effect. If you do not use the `tls://` scheme, then passing `tls` context options has no effect.

Whenever a client connects, it will emit a `connection` event with a connection instance implementing `ConnectionInterface`:

```
$socket->on('connection', function (React\Socket\ConnectionInterface $connection) {
    echo 'Plaintext connection from ' . $connection->getRemoteAddress() . PHP_EOL;

    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

See also the `ServerInterface` for more details.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

> Note that the `SocketServer` class is a concrete implementation for TCP/IP sockets. If you want to typehint in your higher-level protocol implementation, you SHOULD use the generic `ServerInterface` instead.
>
> Changelog v1.9.0: This class has been added with an improved constructor signature as a replacement for the previous `Server` class in order to avoid any ambiguities. The previous name has been deprecated and should not be used anymore.

### Advanced server usage

**TcpServer** The `TcpServer` class implements the `ServerInterface` and is responsible for accepting plaintext TCP/IP connections.

```
$server = new React\Socket\TcpServer(8080);
```

As above, the `$uri` parameter can consist of only a port, in which case the server will default to listening on the localhost address `127.0.0.1`, which means it will not be reachable from outside of this system.

In order to use a random port assignment, you can use the port `0`:

```
$server = new React\Socket\TcpServer(0);
$address = $server->getAddress();
```

In order to change the host the socket is listening on, you can provide an IP address through the first parameter provided to the constructor, optionally preceded by the `tcp://` scheme:

```
$server = new React\Socket\TcpServer('192.168.0.1:8080');
```

If you want to listen on an IPv6 address, you MUST enclose the host in square brackets:

```
$server = new React\Socket\TcpServer('[::1]:8080');
```

If the given URI is invalid, does not contain a port, any other scheme or if it contains a hostname, it will throw an `InvalidArgumentException`:

```
// throws InvalidArgumentException due to missing port
$server = new React\Socket\TcpServer('127.0.0.1');
```

If the given URI appears to be valid, but listening on it fails (such as if port is already in use or port below 1024 may require root access etc.), it will throw a `RuntimeException`:

```
$first = new React\Socket\TcpServer(8080);

// throws RuntimeException because port is already in use
$second = new React\Socket\TcpServer(8080);
```

> Note that these error conditions may vary depending on your system and/or configuration. See the exception message and code for more details about the actual error condition.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

Optionally, you can specify socket context options for the underlying stream socket resource like this:

```
$server = new React\Socket\TcpServer('[::1]:8080', null, array(
    'backlog' => 200,
    'so_reuseport' => true,
    'ipv6_v6only' => true
));
```

> Note that available socket context options, their defaults and effects of changing these may vary depending on your system and/or PHP version. Passing unknown context options has no effect. The `backlog` context option defaults to `511` unless given explicitly.

Whenever a client connects, it will emit a `connection` event with a connection instance implementing `ConnectionInterface`:

```
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    echo 'Plaintext connection from ' . $connection->getRemoteAddress() . PHP_EOL;

    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

See also the `ServerInterface` for more details.

**SecureServer** The `SecureServer` class implements the `ServerInterface` and is responsible for providing a secure TLS (formerly known as SSL) server.

It does so by wrapping a `TcpServer` instance which waits for plaintext TCP/IP connections and then performs a TLS handshake for each connection. It thus requires valid TLS context options, which in its most basic form may look something like this if you're using a PEM encoded certificate file:

```
$server = new React\Socket\TcpServer(8000);
$server = new React\Socket\SecureServer($server, null, array(
    'local_cert' => 'server.pem'
));
```

> Note that the certificate file will not be loaded on instantiation but when an incoming connection initializes its TLS context. This implies that any invalid certificate file paths or contents will only cause an `error` event at a later time.

If your private key is encrypted with a passphrase, you have to specify it like this:

```
$server = new React\Socket\TcpServer(8000);
$server = new React\Socket\SecureServer($server, null, array(
    'local_cert' => 'server.pem',
    'passphrase' => 'secret'
));
```

By default, this server supports TLSv1.0+ and excludes support for legacy SSLv2/SSLv3. As of PHP 5.6+ you can also explicitly choose the TLS version you want to negotiate with the remote side:

```
$server = new React\Socket\TcpServer(8000);
$server = new React\Socket\SecureServer($server, null, array(
    'local_cert' => 'server.pem',
    'crypto_method' => STREAM_CRYPTO_METHOD_TLSv1_2_SERVER
));
```

> Note that available TLS context options, their defaults and effects of changing these may vary depending on your system and/or PHP version. Passing unknown context options has no effect.

Whenever a client completes the TLS handshake, it will emit a `connection` event with a connection instance implementing `ConnectionInterface`:

```
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    echo 'Secure connection from' . $connection->getRemoteAddress() . PHP_EOL;

    $connection->write('hello there!' . PHP_EOL);
```

```
    ...
});
```

Whenever a client fails to perform a successful TLS handshake, it will emit an `error` event and then close the underlying TCP/IP connection:

```
$server->on('error', function (Exception $e) {
    echo 'Error' . $e->getMessage() . PHP_EOL;
});
```

See also the `ServerInterface` for more details.

Note that the `SecureServer` class is a concrete implementation for TLS sockets. If you want to typehint in your higher-level protocol implementation, you SHOULD use the generic `ServerInterface` instead.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

> Advanced usage: Despite allowing any `ServerInterface` as first parameter, you SHOULD pass a `TcpServer` instance as first parameter, unless you know what you're doing. Internally, the `SecureServer` has to set the required TLS context options on the underlying stream resources. These resources are not exposed through any of the interfaces defined in this package, but only through the internal `Connection` class. The `TcpServer` class is guaranteed to emit connections that implement the `ConnectionInterface` and uses the internal `Connection` class in order to expose these underlying resources. If you use a custom `ServerInterface` and its `connection` event does not meet this requirement, the `SecureServer` will emit an `error` event and then close the underlying connection.

**UnixServer** The `UnixServer` class implements the `ServerInterface` and is responsible for accepting connections on Unix domain sockets (UDS).

```
$server = new React\Socket\UnixServer('/tmp/server.sock');
```

As above, the `$uri` parameter can consist of only a socket path or socket path prefixed by the `unix://` scheme.

If the given URI appears to be valid, but listening on it fails (such as if the socket is already in use or the file not accessible etc.), it will throw a `RuntimeException`:

```
$first = new React\Socket\UnixServer('/tmp/same.sock');

// throws RuntimeException because socket is already in use
$second = new React\Socket\UnixServer('/tmp/same.sock');
```

Note that these error conditions may vary depending on your system and/or configuration. In particular, Zend PHP does only report "Unknown error" when the UDS path already exists and can not be bound. You may want to check `is_file()` on the given UDS path to report a more user-friendly error message in this case. See the exception message and code for more details about the actual error condition.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

Whenever a client connects, it will emit a `connection` event with a connection instance implementing `ConnectionInterface`:

```
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    echo 'New connection' . PHP_EOL;

    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

See also the `ServerInterface` for more details.

**LimitingServer** The `LimitingServer` decorator wraps a given `ServerInterface` and is responsible for limiting and keeping track of open connections to this server instance.

Whenever the underlying server emits a `connection` event, it will check its limits and then either - keep track of this connection by adding it to the list of open connections and then forward the `connection` event - or reject (close) the connection when its limits are exceeded and will forward an `error` event instead.

Whenever a connection closes, it will remove this connection from the list of open connections.

```
$server = new React\Socket\LimitingServer($server, 100);
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

See also the second example for more details.

You have to pass a maximum number of open connections to ensure the server will automatically reject (close) connections once this limit is exceeded. In this case, it will emit an `error` event to inform about this and no `connection` event will be emitted.

```php
$server = new React\Socket\LimitingServer($server, 100);
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

You MAY pass a `null` limit in order to put no limit on the number of open connections and keep accepting new connection until you run out of operating system resources (such as open file handles). This may be useful if you do not want to take care of applying a limit but still want to use the `getConnections()` method.

You can optionally configure the server to pause accepting new connections once the connection limit is reached. In this case, it will pause the underlying server and no longer process any new connections at all, thus also no longer closing any excessive connections. The underlying operating system is responsible for keeping a backlog of pending connections until its limit is reached, at which point it will start rejecting further connections. Once the server is below the connection limit, it will continue consuming connections from the backlog and will process any outstanding data on each connection. This mode may be useful for some protocols that are designed to wait for a response message (such as HTTP), but may be less useful for other protocols that demand immediate responses (such as a "welcome" message in an interactive chat).

```php
$server = new React\Socket\LimitingServer($server, 100, true);
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    $connection->write('hello there!' . PHP_EOL);
    ...
});
```

**getConnections()** The `getConnections(): ConnectionInterface[]` method can be used to return an array with all currently active connections.

```php
foreach ($server->getConnection() as $connection) {
    $connection->write('Hi!');
}
```

## Client usage

### ConnectorInterface

The `ConnectorInterface` is responsible for providing an interface for establishing streaming connections, such as a normal TCP/IP connection.

This is the main interface defined in this package and it is used throughout React's vast ecosystem.

Most higher-level components (such as HTTP, database or other networking service clients) accept an instance implementing this interface to create their

TCP/IP connection to the underlying networking service. This is usually done via dependency injection, so it's fairly simple to actually swap this implementation against any other implementation of this interface.

The interface only offers a single method:

**connect()** The `connect(string $uri): PromiseInterface<ConnectionInterface>` method can be used to create a streaming connection to the given remote address.

It returns a Promise which either fulfills with a stream implementing `ConnectionInterface` on success or rejects with an `Exception` if the connection is not successful:

```
$connector->connect('google.com:443')->then(
    function (React\Socket\ConnectionInterface $connection) {
        // connection successfully established
    },
    function (Exception $error) {
        // failed to connect due to $error
    }
);
```

See also `ConnectionInterface` for more details.

The returned Promise MUST be implemented in such a way that it can be cancelled when it is still pending. Cancelling a pending promise MUST reject its value with an `Exception`. It SHOULD clean up any underlying resources and references as applicable:

```
$promise = $connector->connect($uri);

$promise->cancel();
```

### Connector

The `Connector` class is the main class in this package that implements the `ConnectorInterface` and allows you to create streaming connections.

You can use this connector to create any kind of streaming connections, such as plaintext TCP/IP, secure TLS or local Unix connection streams.

It binds to the main event loop and can be used like this:

```
$connector = new React\Socket\Connector();

$connector->connect($uri)->then(function (React\Socket\ConnectionInterface $connection) {
    $connection->write('...');
    $connection->end();
}, function (Exception $e) {
```

```
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
});
```

In order to create a plaintext TCP/IP connection, you can simply pass a host and port combination like this:

```
$connector->connect('www.google.com:80')->then(function (React\Socket\ConnectionInterface $c
    $connection->write('...');
    $connection->end();
});
```

> If you do no specify a URI scheme in the destination URI, it will assume `tcp://` as a default and establish a plaintext TCP/IP connection. Note that TCP/IP connections require a host and port part in the destination URI like above, all other URI components are optional.

In order to create a secure TLS connection, you can use the `tls://` URI scheme like this:

```
$connector->connect('tls://www.google.com:443')->then(function (React\Socket\ConnectionInter
    $connection->write('...');
    $connection->end();
});
```

In order to create a local Unix domain socket connection, you can use the `unix://` URI scheme like this:

```
$connector->connect('unix:///tmp/demo.sock')->then(function (React\Socket\ConnectionInterfac
    $connection->write('...');
    $connection->end();
});
```

> The `getRemoteAddress()` method will return the target Unix domain socket (UDS) path as given to the `connect()` method, including the `unix://` scheme, for example `unix:///tmp/demo.sock`. The `getLocalAddress()` method will most likely return a `null` value as this value is not applicable to UDS connections here.

Under the hood, the `Connector` is implemented as a *higher-level facade* for the lower-level connectors implemented in this package. This means it also shares all of their features and implementation details. If you want to typehint in your higher-level protocol implementation, you SHOULD use the generic `ConnectorInterface` instead.

As of `v1.4.0`, the `Connector` class defaults to using the happy eyeballs algorithm to automatically connect over IPv4 or IPv6 when a hostname is given. This automatically attempts to connect using both IPv4 and IPv6 at the same time (preferring IPv6), thus avoiding the usual problems faced by users with imperfect IPv6 connections or setups. If you want to revert to the old behavior of only

doing an IPv4 lookup and only attempt a single IPv4 connection, you can set up the `Connector` like this:

```
$connector = new React\Socket\Connector(array(
    'happy_eyeballs' => false
));
```

Similarly, you can also affect the default DNS behavior as follows. The `Connector` class will try to detect your system DNS settings (and uses Google's public DNS server `8.8.8.8` as a fallback if unable to determine your system settings) to resolve all public hostnames into underlying IP addresses by default. If you explicitly want to use a custom DNS server (such as a local DNS relay or a company wide DNS server), you can set up the `Connector` like this:

```
$connector = new React\Socket\Connector(array(
    'dns' => '127.0.1.1'
));

$connector->connect('localhost:80')->then(function (React\Socket\ConnectionInterface $connec
    $connection->write('...');
    $connection->end();
});
```

If you do not want to use a DNS resolver at all and want to connect to IP addresses only, you can also set up your `Connector` like this:

```
$connector = new React\Socket\Connector(array(
    'dns' => false
));

$connector->connect('127.0.0.1:80')->then(function (React\Socket\ConnectionInterface $connec
    $connection->write('...');
    $connection->end();
});
```

Advanced: If you need a custom DNS `React\Dns\Resolver\ResolverInterface` instance, you can also set up your `Connector` like this:

```
$dnsResolverFactory = new React\Dns\Resolver\Factory();
$resolver = $dnsResolverFactory->createCached('127.0.1.1');

$connector = new React\Socket\Connector(array(
    'dns' => $resolver
));

$connector->connect('localhost:80')->then(function (React\Socket\ConnectionInterface $connec
    $connection->write('...');
    $connection->end();
});
```

By default, the `tcp://` and `tls://` URI schemes will use timeout value that respects your `default_socket_timeout` ini setting (which defaults to 60s). If you want a custom timeout value, you can simply pass this like this:

```
$connector = new React\Socket\Connector(array(
    'timeout' => 10.0
));
```

Similarly, if you do not want to apply a timeout at all and let the operating system handle this, you can pass a boolean flag like this:

```
$connector = new React\Socket\Connector(array(
    'timeout' => false
));
```

By default, the `Connector` supports the `tcp://`, `tls://` and `unix://` URI schemes. If you want to explicitly prohibit any of these, you can simply pass boolean flags like this:

```
// only allow secure TLS connections
$connector = new React\Socket\Connector(array(
    'tcp' => false,
    'tls' => true,
    'unix' => false,
));

$connector->connect('tls://google.com:443')->then(function (React\Socket\ConnectionInterface
    $connection->write('...');
    $connection->end();
});
```

The `tcp://` and `tls://` also accept additional context options passed to the underlying connectors. If you want to explicitly pass additional context options, you can simply pass arrays of context options like this:

```
// allow insecure TLS connections
$connector = new React\Socket\Connector(array(
    'tcp' => array(
        'bindto' => '192.168.0.1:0'
    ),
    'tls' => array(
        'verify_peer' => false,
        'verify_peer_name' => false
    ),
));

$connector->connect('tls://localhost:443')->then(function (React\Socket\ConnectionInterface
    $connection->write('...');
    $connection->end();
```

```
});
```

By default, this connector supports TLSv1.0+ and excludes support for legacy SSLv2/SSLv3. As of PHP 5.6+ you can also explicitly choose the TLS version you want to negotiate with the remote side:

```
$connector = new React\Socket\Connector(array(
    'tls' => array(
        'crypto_method' => STREAM_CRYPTO_METHOD_TLSv1_2_CLIENT
    )
));
```

> For more details about context options, please refer to the PHP documentation about socket context options and SSL context options.

Advanced: By default, the `Connector` supports the `tcp://`, `tls://` and `unix://` URI schemes. For this, it sets up the required connector classes automatically. If you want to explicitly pass custom connectors for any of these, you can simply pass an instance implementing the `ConnectorInterface` like this:

```
$dnsResolverFactory = new React\Dns\Resolver\Factory();
$resolver = $dnsResolverFactory->createCached('127.0.1.1');
$tcp = new React\Socket\HappyEyeBallsConnector(null, new React\Socket\TcpConnector(), $resol

$tls = new React\Socket\SecureConnector($tcp);

$unix = new React\Socket\UnixConnector();

$connector = new React\Socket\Connector(array(
    'tcp' => $tcp,
    'tls' => $tls,
    'unix' => $unix,

    'dns' => false,
    'timeout' => false,
));

$connector->connect('google.com:80')->then(function (React\Socket\ConnectionInterface $conne
    $connection->write('...');
    $connection->end();
});
```

> Internally, the `tcp://` connector will always be wrapped by the DNS resolver, unless you disable DNS like in the above example. In this case, the `tcp://` connector receives the actual hostname instead of only the resolved IP address and is thus responsible for performing the lookup. Internally, the automatically created `tls://` connector will always wrap the underlying `tcp://` connector for establishing the underlying plaintext TCP/IP connection before enabling secure

TLS mode. If you want to use a custom underlying `tcp://` connector for secure TLS connections only, you may explicitly pass a `tls://` connector like above instead. Internally, the `tcp://` and `tls://` connectors will always be wrapped by `TimeoutConnector`, unless you disable timeouts like in the above example.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

Changelog v1.9.0: The constructur signature has been updated to take the optional `$context` as the first parameter and the optional `$loop` as a second argument. The previous signature has been deprecated and should not be used anymore.

```
// constructor signature as of v1.9.0
$connector = new React\Socket\Connector(array $context = [], ?LoopInterface $loop = nul

// legacy constructor signature before v1.9.0
$connector = new React\Socket\Connector(?LoopInterface $loop = null, array $context = [
```

### Advanced client usage

**TcpConnector** The `TcpConnector` class implements the `ConnectorInterface` and allows you to create plaintext TCP/IP connections to any IP-port-combination:

```
$tcpConnector = new React\Socket\TcpConnector();

$tcpConnector->connect('127.0.0.1:80')->then(function (React\Socket\ConnectionInterface $con
    $connection->write('...');
    $connection->end();
});
```

See also the examples.

Pending connection attempts can be cancelled by cancelling its pending promise like so:

```
$promise = $tcpConnector->connect('127.0.0.1:80');

$promise->cancel();
```

Calling `cancel()` on a pending promise will close the underlying socket resource, thus cancelling the pending TCP/IP connection, and reject the resulting promise.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a

`null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

You can optionally pass additional socket context options to the constructor like this:

```
$tcpConnector = new React\Socket\TcpConnector(null, array(
    'bindto' => '192.168.0.1:0'
));
```

Note that this class only allows you to connect to IP-port-combinations. If the given URI is invalid, does not contain a valid IP address and port or contains any other scheme, it will reject with an `InvalidArgumentException`:

If the given URI appears to be valid, but connecting to it fails (such as if the remote host rejects the connection etc.), it will reject with a `RuntimeException`.

If you want to connect to hostname-port-combinations, see also the following chapter.

> Advanced usage: Internally, the `TcpConnector` allocates an empty *context* resource for each stream resource. If the destination URI contains a `hostname` query parameter, its value will be used to set up the TLS peer name. This is used by the `SecureConnector` and `DnsConnector` to verify the peer name and can also be used if you want a custom TLS peer name.

**HappyEyeBallsConnector** The `HappyEyeBallsConnector` class implements the `ConnectorInterface` and allows you to create plaintext TCP/IP connections to any hostname-port-combination. Internally it implements the happy eyeballs algorithm from `RFC6555` and `RFC8305` to support IPv6 and IPv4 hostnames.

It does so by decorating a given `TcpConnector` instance so that it first looks up the given domain name via DNS (if applicable) and then establishes the underlying TCP/IP connection to the resolved target IP address.

Make sure to set up your DNS resolver and underlying TCP connector like this:

```
$dnsResolverFactory = new React\Dns\Resolver\Factory();
$dns = $dnsResolverFactory->createCached('8.8.8.8');

$dnsConnector = new React\Socket\HappyEyeBallsConnector(null, $tcpConnector, $dns);

$dnsConnector->connect('www.google.com:80')->then(function (React\Socket\ConnectionInterface
    $connection->write('...');
    $connection->end();
});
```

See also the examples.

Pending connection attempts can be cancelled by cancelling its pending promise like so:

```
$promise = $dnsConnector->connect('www.google.com:80');

$promise->cancel();
```

Calling `cancel()` on a pending promise will cancel the underlying DNS lookups and/or the underlying TCP/IP connection(s) and reject the resulting promise.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

> Advanced usage: Internally, the `HappyEyeBallsConnector` relies on a `Resolver` to look up the IP addresses for the given hostname. It will then replace the hostname in the destination URI with this IP's and append a `hostname` query parameter and pass this updated URI to the underlying connector. The Happy Eye Balls algorithm describes looking the IPv6 and IPv4 address for the given hostname so this connector sends out two DNS lookups for the A and AAAA records. It then uses all IP addresses (both v6 and v4) and tries to connect to all of them with a 50ms interval in between. Alterating between IPv6 and IPv4 addresses. When a connection is established all the other DNS lookups and connection attempts are cancelled.

**DnsConnector** The `DnsConnector` class implements the `ConnectorInterface` and allows you to create plaintext TCP/IP connections to any hostname-port-combination.

It does so by decorating a given `TcpConnector` instance so that it first looks up the given domain name via DNS (if applicable) and then establishes the underlying TCP/IP connection to the resolved target IP address.

Make sure to set up your DNS resolver and underlying TCP connector like this:

```
$dnsResolverFactory = new React\Dns\Resolver\Factory();
$dns = $dnsResolverFactory->createCached('8.8.8.8');

$dnsConnector = new React\Socket\DnsConnector($tcpConnector, $dns);

$dnsConnector->connect('www.google.com:80')->then(function (React\Socket\ConnectionInterface
    $connection->write('...');
    $connection->end();
});
```

See also the examples.

Pending connection attempts can be cancelled by cancelling its pending promise like so:

```
$promise = $dnsConnector->connect('www.google.com:80');

$promise->cancel();
```

Calling `cancel()` on a pending promise will cancel the underlying DNS lookup and/or the underlying TCP/IP connection and reject the resulting promise.

> Advanced usage: Internally, the `DnsConnector` relies on a `React\Dns\Resolver\ResolverInterface` to look up the IP address for the given hostname. It will then replace the hostname in the destination URI with this IP and append a `hostname` query parameter and pass this updated URI to the underlying connector. The underlying connector is thus responsible for creating a connection to the target IP address, while this query parameter can be used to check the original hostname and is used by the `TcpConnector` to set up the TLS peer name. If a `hostname` is given explicitly, this query parameter will not be modified, which can be useful if you want a custom TLS peer name.

**SecureConnector** The `SecureConnector` class implements the `ConnectorInterface` and allows you to create secure TLS (formerly known as SSL) connections to any hostname-port-combination.

It does so by decorating a given `DnsConnector` instance so that it first creates a plaintext TCP/IP connection and then enables TLS encryption on this stream.

```
$secureConnector = new React\Socket\SecureConnector($dnsConnector);

$secureConnector->connect('www.google.com:443')->then(function (React\Socket\ConnectionInter
    $connection->write("GET / HTTP/1.0\r\nHost: www.google.com\r\n\r\n");
    ...
});
```

See also the examples.

Pending connection attempts can be cancelled by cancelling its pending promise like so:

```
$promise = $secureConnector->connect('www.google.com:443');

$promise->cancel();
```

Calling `cancel()` on a pending promise will cancel the underlying TCP/IP connection and/or the SSL/TLS negotiation and reject the resulting promise.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a

`null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

You can optionally pass additional SSL context options to the constructor like this:

```
$secureConnector = new React\Socket\SecureConnector($dnsConnector, null, array(
    'verify_peer' => false,
    'verify_peer_name' => false
));
```

By default, this connector supports TLSv1.0+ and excludes support for legacy SSLv2/SSLv3. As of PHP 5.6+ you can also explicitly choose the TLS version you want to negotiate with the remote side:

```
$secureConnector = new React\Socket\SecureConnector($dnsConnector, null, array(
    'crypto_method' => STREAM_CRYPTO_METHOD_TLSv1_2_CLIENT
));
```

> Advanced usage: Internally, the `SecureConnector` relies on setting up the required *context options* on the underlying stream resource. It should therefor be used with a `TcpConnector` somewhere in the connector stack so that it can allocate an empty *context* resource for each stream resource and verify the peer name. Failing to do so may result in a TLS peer name mismatch error or some hard to trace race conditions, because all stream resources will use a single, shared *default context* resource otherwise.

**TimeoutConnector** The `TimeoutConnector` class implements the `ConnectorInterface` and allows you to add timeout handling to any existing connector instance.

It does so by decorating any given `ConnectorInterface` instance and starting a timer that will automatically reject and abort any underlying connection attempt if it takes too long.

```
$timeoutConnector = new React\Socket\TimeoutConnector($connector, 3.0);

$timeoutConnector->connect('google.com:80')->then(function (React\Socket\ConnectionInterface
    // connection succeeded within 3.0 seconds
});
```

See also any of the examples.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

Pending connection attempts can be cancelled by cancelling its pending promise like so:

```
$promise = $timeoutConnector->connect('google.com:80');

$promise->cancel();
```

Calling `cancel()` on a pending promise will cancel the underlying connection attempt, abort the timer and reject the resulting promise.

**UnixConnector** The `UnixConnector` class implements the `ConnectorInterface` and allows you to connect to Unix domain socket (UDS) paths like this:

```
$connector = new React\Socket\UnixConnector();

$connector->connect('/tmp/demo.sock')->then(function (React\Socket\ConnectionInterface $conn
    $connection->write("HELLO\n");
});
```

Connecting to Unix domain sockets is an atomic operation, i.e. its promise will settle (either resolve or reject) immediately. As such, calling `cancel()` on the resulting promise has no effect.

> The `getRemoteAddress()` method will return the target Unix domain socket (UDS) path as given to the `connect()` method, prepended with the `unix://` scheme, for example `unix:///tmp/demo.sock`. The `getLocalAddress()` method will most likely return a `null` value as this value is not applicable to UDS connections here.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

**FixedUriConnector** The `FixedUriConnector` class implements the `ConnectorInterface` and decorates an existing Connector to always use a fixed, preconfigured URI.

This can be useful for consumers that do not support certain URIs, such as when you want to explicitly connect to a Unix domain socket (UDS) path instead of connecting to a default address assumed by an higher-level API:

```
$connector = new React\Socket\FixedUriConnector(
    'unix:///var/run/docker.sock',
    new React\Socket\UnixConnector()
);

// destination will be ignored, actually connects to Unix domain socket
$promise = $connector->connect('localhost:80');
```

## Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version:

```
composer require react/socket:^1.16
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use the latest supported PHP version* for this project, partly due to its vast performance improvements and partly because legacy PHP versions require several workarounds as described below.

Secure TLS connections received some major upgrades starting with PHP 5.6, with the defaults now being more secure, while older versions required explicit context options. This library does not take responsibility over these context options, so it's up to consumers of this library to take care of setting appropriate context options as described above.

PHP < 7.3.3 (and PHP < 7.2.15) suffers from a bug where feof() might block with 100% CPU usage on fragmented TLS records. We try to work around this by always consuming the complete receive buffer at once to avoid stale data in TLS buffers. This is known to work around high CPU usage for well-behaving peers, but this may cause very large data chunks for high throughput scenarios. The buggy behavior can still be triggered due to network I/O buffers or malicious peers on affected versions, upgrading is highly recommended.

PHP < 7.1.4 (and PHP < 7.0.18) suffers from a bug when writing big chunks of data over TLS streams at once. We try to work around this by limiting the write chunk size to 8192 bytes for older PHP versions only. This is only a work-around and has a noticable performance penalty on affected versions.

This project also supports running on HHVM. Note that really old HHVM < 3.8 does not support secure TLS connections, as it lacks the required `stream_socket_enable_crypto()` function. As such, trying to create a secure TLS connections on affected versions will return a rejected promise instead. This issue is also covered by our test suite, which will skip related tests on affected versions.

## Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

The test suite also contains a number of functional integration tests that rely on a stable internet connection. If you do not want to run these, they can simply be skipped like this:

```
vendor/bin/phpunit --exclude-group internet
```

## License

MIT, see LICENSE file.